

Módulo 03 - Kits e Regras de Seleção

Objetivo

Entender como o catálogo evolui para vender combinações de brinquedos: kits prontos, kits personalizáveis e validações de seleção calculadas no backend.

Commits Estudados

- `ada8783` - adiciona `KitFesta` e `ItemKitFesta`.
- `5c393e5` - adiciona configuração de kits personalizáveis.
- `93e1ac8` - valida seleção para kits personalizáveis.

Aula 1 - Kit Pronto Como Composição

O commit `ada8783` cria `KitFesta` e `ItemKitFesta`.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/services.py`
- `backend/catalogo/views.py`
- `backend/catalogo/urls.py`
- `backend/catalogo/tests.py`

Modelagem principal:

- `KitFesta` tem nome, descrição, preço, ativo e ordem.
- `ItemKitFesta` liga um kit a um brinquedo com quantidade.
- A constraint impede repetir o mesmo brinquedo no mesmo kit.

No estado atual:

```
models.UniqueConstraint(  
    fields=["kit", "brinquedo"],
```

```
name="catalogo_item_kit_festa_brinquedo_unico",  
)
```

Decisão de produto visível no código:

- Kit pronto é um produto de catálogo com composição fixa.
- O preço do kit é próprio, não soma automática dos brinquedos.
- A composição é lida pelo cliente, mas alterada apenas por admin.

Testes para estudar:

- `test_usuario_anonimo_lista_apenas_kits_ativos`
- `test_api_publica_retorna_itens_do_kit`
- `test_nao_permite_mesmo_brinquedo_duas_vezes_no_mesmo_kit`

Aula 2 - Kit Personalizável Como Configuração

O commit `5c393e5` cria a base dos kits personalizáveis.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/admin.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/services.py`
- `backend/catalogo/views.py`

Entidades introduzidas:

- `ConfiguracaoKitPersonalizavel`
- `RegraCategoriaKitPersonalizavel`

Campos importantes:

- `quantidade_minima_brinquedos`
- `quantidade_maxima_brinquedos`
- `modo_elegibilidade`
- `categorias_permitidas`
- `brinquedos_permitidos`
- `preco_base`

O modelo mostra três formas de elegibilidade:

- por categorias;
- por brinquedos específicos;
- por união entre categorias e brinquedos.

Ponto de domínio:

- O produto não é apenas "monte qualquer coisa".
- O backend define quais brinquedos podem entrar e quais limites existem.

Testes para estudar:

- `test_configuracao_rejeita_quantidade_minima_maior_que_maxima`
- `test_regra_categoria_rejeita_quantidade_minima_maior_que_maxima`
- `test_api_publica_retorna_brinquedos_elegiveis_no_modos_categorias`
- `test_api_publica_retorna_uniao_sem_duplicidade_no_modos_categorias_e_brinquedos`

Aula 3 - Validação de Seleção no Backend

O commit `93e1ac8` adiciona a validação de seleção de kit personalizável.

Código para ler:

- `backend/catalogo/services.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/views.py`
- `backend/catalogo/tests.py`

Endpoint criado:

- `POST /api/kits-personalizaveis/<id>/validar-selecao/`

Responsabilidades reais:

- rejeitar brinquedo fora da configuração;
- rejeitar quantidade total abaixo do mínimo;
- rejeitar quantidade total acima do máximo;
- rejeitar brinquedo duplicado no payload;
- validar regra mínima e máxima por categoria;
- calcular preço estimado no backend.

Trecho conceitual no service:

```
quantidade_total = sum(item["quantidade"] for item in itens)
if quantidade_total < configuracao.quantidade_minima_brinquedos:
    raise serializers.ValidationError(...)
```

Ponto crítico:

- O frontend pode ajudar a montar a seleção, mas não é fonte de verdade.
- O preço estimado volta do backend.

Teste que deixa isso claro:

- `test_validar_selecao_retorna_resumo_com_preco_estimado_do_backend`

Esse teste envia um valor forjado no payload e valida que o backend calcula o preço correto.

Aula 4 - Separação Entre Seleção e Disponibilidade

Um detalhe importante aparece na evolução posterior do código: validar seleção não é o mesmo que garantir estoque no período.

No commit `93e1ac8`, o foco é composição e preço.

No commit `5ca95fe`, a disponibilidade por período passa a considerar unidades livres e reservas conflitantes.

Essa separação é boa:

- seleção responde "este kit é válido?";
- disponibilidade responde "há unidades para esta data?";
- reserva responde "o admin bloqueou unidades físicas para este pedido?".

Revisão do Módulo

Você deve conseguir responder:

- Qual diferença entre `KitFesta` e `ConfiguracaoKitPersonalizavel`?
- Por que `ItemKitFesta` tem constraint única?
- Onde o backend calcula preço de kit personalizado?
- Por que validar seleção não pode criar reserva?
- Que teste prova que o frontend não define preço?

Exercício Prático

Abra `git show 93e1ac8` e siga o fluxo:

1. serializer valida formato do payload;
2. view chama service;
3. service calcula resumo;
4. teste prova regra de negócio.

Depois compare com `git show 5ca95fe` para ver como disponibilidade foi tratada depois.